

Clean Room Reproducible Build Process

Module: pqc_crypto_module v0.1.0 **Date:** 2026-04-28

Disclaimer: This package does not imply FIPS 140-3 validation or CMVP certification.

1. Purpose

FIPS 140-3 lifecycle assurance requires evidence that the module binary can be reproduced from source. This document defines a clean-room build process that:

- 1. Starts from a known-good base environment (Docker or VM)
- 2. Pins all toolchain and dependency versions
- 3. Builds offline after an initial fetch
- 4. Hashes the resulting artifacts
- 5. Compares hashes across independent builds

The reference script is `scripts/clean_room_build.sh`.

2. Prerequisites

Component	Pinned version	Source
Base image	rust:nightly-2026-04-01-slim-bookworm	Docker Hub official Rust image
Rust toolchain	nightly-2026-04-01	Installed via rustup in image
Target	x86_64-unknown-linux-gnu	Default for Debian Bookworm
Cargo.lock	Committed in repository	Must not be modified during build
OS packages	build-essential, pkg-config, libclang-dev	Required for C FFI (pqcrypto-internals)

3. Build Environment Setup

3.1 Dockerfile

```
FROM rust:nightly-2026-04-01-slim-bookworm AS builder

# Install system dependencies for C FFI compilation
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    pkg-config \
    libclang-dev \
    && rm -rf /var/lib/apt/lists/*

# Pin rustup components
RUN rustup component add rustfmt clippy

# Create build user (non-root)
RUN useradd -m builder
USER builder
WORKDIR /home/builder/src

# Copy source
COPY --chown=builder:builder . .

# Verify Cargo.lock is present and unchanged
RUN sha256sum Cargo.lock > /tmp/lock_hash_before.txt

# Fetch dependencies (online phase)
RUN cargo fetch --locked

# Verify Cargo.lock was not modified
RUN sha256sum -c /tmp/lock_hash_before.txt

# Build offline (no network access after this point)
RUN cargo build --release --offline --locked \
    -p pqc_crypto_module 2>&1 | tee /tmp/build.log

# Hash artifacts
RUN sha256sum target/release/libpqc_crypto_module.rlib > /tmp/
    artifact_hashes.txt && \
    sha256sum target/release/deps/libpqc_crypto_module-*.rlib >> /
    tmp/artifact_hashes.txt 2>/dev/null || true

# Copy out results
RUN cp /tmp/artifact_hashes.txt /home/builder/artifact_hashes.txt
RUN cp /tmp/build.log /home/builder/build.log
```

3.2 Alternative: VM-based build

For environments where Docker is not acceptable:

1. Provision a fresh Debian Bookworm VM (minimal install)

2. Install Rust nightly-2026-04-01 via rustup
 3. Copy source tarball and verify its SHA-256
 4. Follow the same fetch -> offline build -> hash workflow
-

4. Build Procedure

Step 1: Prepare source archive

```
# From the repository root
git archive --format=tar.gz --prefix=ppc_crypto_module/ HEAD \
    -o ppc_crypto_module_source.tar.gz

# Record source hash
sha256sum ppc_crypto_module_source.tar.gz > source_hash.txt
```

Step 2: Build (Run 1)

```
# Build the Docker image
docker build -t ppc-clean-build:run1 -f Dockerfile.clean-room .

# Extract artifacts
docker create --name run1 ppc-clean-build:run1
docker cp run1:/home/builder/artifact_hashes.txt ./run1_hashes.txt
docker cp run1:/home/builder/build.log ./run1_build.log
docker rm run1
```

Step 3: Build (Run 2 — independent)

```
# Clean Docker cache to force independent build
docker builder prune -f

# Rebuild
docker build --no-cache -t ppc-clean-build:run2 -f
    Dockerfile.clean-room .

# Extract artifacts
docker create --name run2 ppc-clean-build:run2
docker cp run2:/home/builder/artifact_hashes.txt ./run2_hashes.txt
docker cp run2:/home/builder/build.log ./run2_build.log
docker rm run2
```

Step 4: Compare

```
diff run1_hashes.txt run2_hashes.txt
if [ $? -eq 0 ]; then
    echo "REPRODUCIBLE: Artifact hashes match across independent
        builds."
else
```

```
    echo "NOT REPRODUCIBLE: Artifact hashes differ. Investigation
        required."
    diff -u run1_hashes.txt run2_hashes.txt
fi
```

5. Known Reproducibility Challenges

Challenge	Mitigation
Rust compiler embeds timestamps or paths	Use <code>--remap-path-prefix</code> in <code>RUSTFLAGS</code> to normalize paths. Set <code>SOURCE_DATE_EPOCH</code> for timestamp reproducibility.
C FFI compilation (pqcrypto-internals) may vary	Pin exact system package versions in Dockerfile. Use identical base image.
Link order non-determinism	Use <code>RUSTFLAGS="-C link-arg=-Wl,--sort-section=name"</code> on Linux.
.rlib metadata differences	Compare the object code sections, not the full .rlib (which contains metadata that may include build paths).
Nightly compiler instability	Pin to a specific nightly date, not latest.

Recommended RUSTFLAGS

```
export RUSTFLAGS="\
    --remap-path-prefix=/home/builder/src=pqc_crypto_module \
    -C link-arg=-Wl,--sort-section=name \
    -C debuginfo=0"
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
```

6. Reference Script

`scripts/clean_room_build.sh` automates Steps 1-4:

```
#!/usr/bin/env bash
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
REPO_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"

echo "=== Clean Room Build ==="
echo "Source: $REPO_ROOT"
echo "Date: $(date -u +%Y-%m-%dT%H:%M:%SZ)"

cd "$REPO_ROOT"
```

```

# Step 1: Source archive
git archive --format=tar.gz --prefix=pqc_crypto_module/ HEAD \
-o /tmp/pqc_source.tar.gz
echo "Source SHA-256: $(sha256sum /tmp/pqc_source.tar.gz)"

# Step 2: Build Run 1
echo "--- Run 1 ---"
docker build --no-cache -t pqc-clean:run1 -f Dockerfile.clean-
room .
CID1=$(docker create pqc-clean:run1)
docker cp "$CID1:/home/builder/artifact_hashes.txt" /tmp/
run1_hashes.txt
docker rm "$CID1"

# Step 3: Build Run 2
echo "--- Run 2 ---"
docker builder prune -f
docker build --no-cache -t pqc-clean:run2 -f Dockerfile.clean-
room .
CID2=$(docker create pqc-clean:run2)
docker cp "$CID2:/home/builder/artifact_hashes.txt" /tmp/
run2_hashes.txt
docker rm "$CID2"

# Step 4: Compare
echo "--- Comparison ---"
if diff /tmp/run1_hashes.txt /tmp/run2_hashes.txt > /dev/null
2>&1; then
    echo "PASS: Reproducible build confirmed."
    cat /tmp/run1_hashes.txt
else
    echo "FAIL: Builds are not reproducible."
    diff -u /tmp/run1_hashes.txt /tmp/run2_hashes.txt
    exit 1
fi

```

7. Evidence Artifacts

After a successful reproducible build, archive the following:

Artifact	Purpose
source_hash.txt	SHA-256 of the source archive
Cargo.lock	Exact dependency versions
run1_hashes.txt	Artifact hashes from build 1
run2_hashes.txt	Artifact hashes from build 2
run1_build.log	Full build output from run 1
run2_build.log	Full build output from run 2
Dockerfile.clean-room	Build environment definition

Artifact	Purpose
Rust toolchain version output	<code>rustc --version --verbose</code>

8. Current Status

Not yet executed. The build process is defined but no comparison artifacts have been produced. See finding F-011 in `MOCK_AUDIT_REPORT.md`.

End of clean room build process.